Planejamento Lógico TaskTracker – Davi Oliveira Silva

Curso/Disciplina: Ciência da Computação- Bootcamp II
Polo/Turma: Turma A - 0726
E-mail institucional: davioli@sempreceub.com
Data: 13 de setembro de 2026

1. Apresentação e entendimento do problema

1.1 Identificação

Meu nome é Davi Oliveira, estou cursando Ciência da Computação, pertencente à turma A. Este documento detalha a arquitetura e a lógica inicial do TaskTracker, um sistema desenvolvido para facilitar o gerenciamento de tarefas do dia a dia.

1.2 O problema a ser resolvido

A rotina moderna exige que estudantes e profissionais lidem com múltiplos compromissos simultâneos. Quando essas responsabilidades são gerenciadas de forma desestruturada — usando apenas a memória, anotações físicas espalhadas ou ferramentas inadequadas —, o resultado frequentemente envolve atrasos, esquecimento de entregas cruciais e confusão na priorização. Falta um ambiente único e focado para registrar, classificar e acompanhar o progresso dessas pendências.

1.3 A solução proposta: TaskTracker

O TaskTracker será um aplicativo executado via linha de comando (CLI), a ser programado em Python. Ele atuará como um centralizador de produtividade, oferecendo ao usuário controle total sobre suas demandas.
Nesta etapa inicial, o sistema será capaz de:

- Registrar novas tarefas contendo título, descrição, nível de prioridade e prazo.
- Exibir um panorama completo de todas as tarefas cadastradas.
- Diferenciar claramente atividades pendentes das já finalizadas.
- Atualizar o status de tarefas específicas para "concluída".
- Fornecer feedback visual adequado (mensagens de erro, validação e sucesso).

Exemplo prático: Um usuário insere a tarefa "Entregar relatório do Bootcamp", sinaliza com prioridade "Alta" e estabelece a data de entrega. Assim que o trabalho for enviado, ele acessa o sistema e muda o status da tarefa para concluída, limpando sua lista de pendências.

2. Decomposição do problema e dados do sistema

2.1 Entradas de dados

Para interagir com o sistema, o usuário fornecerá os seguintes dados:

Dado	Descrição	Exemplo
Título	Ação principal a ser realizada	Ler artigo sobre Git
Descrição	Detalhes ou contexto extra (opcional)	Capítulos 1 a 3 do livro base
Prioridade	Classificação de urgência da tarefa	Baixa, Média ou Alta
Data	Prazo estimado no formato padrão	25/09/2026
Identificador (ID)	Número único gerado pelo sistema para buscas	5

Nota: O usuário não precisa informar o "Status" durante o cadastro. O sistema assumirá automaticamente que qualquer tarefa recém-criada possui o status "Pendente".

2.2 Saídas de dados

O sistema devolverá ao usuário as seguintes respostas:

Saída	Finalidade
Painel de Tarefas	Mostra todas as tarefas com seus respectivos IDs, títulos, prioridades, prazos e status.
Avisos de Sucesso	Confirma que uma tarefa foi salva ou concluída corretamente.
Avisos de Erro/Validação	Alerta o usuário caso ele digite informações em branco, formatos incorretos ou opções

Saída	Finalidade
	inexistentes.
Alerta de Lista Vazia	Informa amigavelmente que o banco de tarefas está limpo no momento.
Painel de Estatísticas	Exibe o contador numérico de quantas tarefas estão pendentes versus concluídas.

Exemplo de exibição:

```
[5] Ler artigo sobre Git | Prioridade: Baixa | Data: 25/09/2026 | Status: Pendente
```

2.3 Regras de Negócio (Restrições)

1. O preenchimento do Título é obrigatório; o sistema rejeitará entradas em branco ou contendo apenas espaços.
2. O campo de Prioridade é estrito e só validará as palavras: Alta, Média ou Baixa.
3. Toda inserção de nova tarefa gera obrigatoriamente um status "Pendente".
4. A ação de "Concluir tarefa" exige um Identificador (ID) válido e existente no banco de dados.
5. Prazos informados devem seguir rigorosamente a máscara dd/mm/aaaa.
6. A exibição (listagem) deve ser formatada e fácil de ler.
7. O sistema nunca deve quebrar ou ficar em branco ao listar tarefas inexistentes; ele deve exibir o aviso explícito: "Nenhuma tarefa cadastrada".

3. Receita lógica do sistema (algoritmo em português)

3.1 Estrutura Inicial

1. Iniciar a execução do aplicativo TaskTracker.
2. Gerar uma base de dados temporária (lista vazia) para as tarefas.
3. Configurar o contador do ID inicial começando em 1.
4. Exibir o Menu Principal: [1] Cadastrar, [2] Listar, [3] Concluir e [4] Sair.
5. Ler o comando do usuário.
6. Manter o programa rodando em loop até que o usuário escolha a opção "Sair".

3.2 Algoritmo de Cadastro

1. Usuário aciona a funcionalidade "Cadastrar tarefa".
2. O programa solicita o Título.
3. Se o Título estiver em branco, exibir aviso de erro e pedir a digitação novamente.
4. O programa solicita a Descrição.
5. O programa solicita a Prioridade.
6. Se a Prioridade for diferente de Alta, Média ou Baixa, avisar o erro e pedir novamente.
7. O programa solicita a Data do prazo.
8. Se a Data não seguir o formato dd/mm/aaaa, recusar a entrada e pedir novamente.
9. Empacotar Título, Descrição, Prioridade e Data junto com o status "Pendente" e o ID atual.
10. Salvar o pacote na lista de tarefas.
11. Adicionar +1 ao contador de IDs.
12. Imprimir mensagem de "Tarefa registrada com sucesso" e voltar ao Menu.

3.3 Algoritmo de Visualização

1. Usuário aciona a funcionalidade "Listar tarefas".
2. O programa checa o tamanho da lista de tarefas.
3. Se o tamanho for zero, imprimir "Nenhuma tarefa cadastrada".
4. Caso contrário, iterar sobre a lista, imprimindo ID, Título, Prioridade, Data e Status de cada item.
5. Somar quantas possuem status Pendente e quantas possuem status Concluída.
6. Imprimir o resumo estatístico na tela.
7. Retornar ao Menu Principal.

3.4 Algoritmo de Conclusão

1. Usuário aciona a funcionalidade "Concluir tarefa".
2. O programa verifica se há itens na lista. Se estiver vazia, avisa que não há nada a concluir e volta ao Menu.
3. O programa imprime rapidamente as tarefas pendentes para consulta.
4. Solicita ao usuário que digite o ID da tarefa desejada.
5. Busca o ID digitado dentro da base de tarefas.
6. Se não localizar, avisa "ID inválido/não encontrado".
7. Se localizar e a tarefa estiver "Pendente", muda a variável de status para "Concluída" e exibe mensagem de sucesso.
8. Se a tarefa já estiver "Concluída", informa que o item já havia sido finalizado.
9. Retornar ao Menu Principal.

3.5 Algoritmo de Encerramento

1. Usuário digita a opção "Sair".
2. O sistema imprime "Finalizando o TaskTracker. Até logo!".
3. Encerra a execução do código.

4. Mapeamento arquitetural para Git e GitHub

4.1 Estrutura de Diretórios

```
tasktracker-projeto/
├── README.md
├── .gitignore
├── docs/
│   └── planejamento_logico.md
└── src/
    └── main.py
```

4.2 Finalidade dos Componentes

- **tasktracker-projeto/**: Raiz do repositório onde todo o ecossistema do projeto viverá.
- **README.md**: Cartão de visitas do projeto no GitHub. Conterá um manual de instruções, dependências e a explicação geral do sistema.
- **.gitignore**: Filtro de segurança que impedirá o upload de arquivos indesejados (como cache do Python ou variáveis de ambiente).
- **docs/**: Diretório exclusivo para manuais e diagramas.
- **docs/planejamento_logico.md**: Local definitivo deste planejamento inicial em formato Markdown.
- **src/**: A pasta do código-fonte, onde a programação real acontece.
- **src/main.py**: Arquivo executável central (entrypoint) que abrigará as lógicas delineadas na seção 3.

4.3 Trilhas de Versionamento (Commits)

Para manter o projeto organizado na Fase 2, os commits seguirão um padrão granular, isolando implementações:

- docs: adiciona escopo e planejamento lógico do projeto
- feat: estrutura menu interativo no terminal
- feat: adiciona lógica de cadastro e validação de dados
- feat: implementa listagem e sumarização de tarefas
- feat: cria função de conclusão de pendências

4.4 Plano de Aprendizado Prático

Durante a Fase 2, meu objetivo principal é consolidar o uso prático de ferramentas de versionamento de forma integrada à programação. Espero dominar o ciclo de vida de um repositório, compreendendo na prática como os comandos git add, git commit e git push operam para proteger meu código contra perdas. Pretendo aprender a escrever mensagens de

commit significativas que documentem corretamente a evolução do software, além de compreender como o GitHub atua como um portfólio remoto que facilita a distribuição do sistema para outros desenvolvedores executarem meu código a partir do README.

5. Link da apresentação em vídeo

Link do vídeo no YouTube: https://youtu.be/chxyhQXj2TA